

Computer Network Assignment (2)

1.

- a) How is a UDP socket fully identified? What about a TCP socket? What is the difference between the full identification of both sockets?
- b) Describe why an application developer might choose to run an application over UDP rather than TCP.
- c) Why is it that voice and video traffic is often sent over TCP rather than UDP in today's Internet?

a) How is a UDP socket fully identified? What about a TCP socket? What is the difference?

UDP socket:

A UDP socket is fully identified by two things:

- The destination IP address
- The destination port number

UDP does not create a connection, so it does not need to track the source.

TCP socket:

A TCP socket is fully identified by four things:

- Source IP address
- Source port number
- Destination IP address
- Destination port number

TCP needs all four because it sets up a reliable connection between exactly two devices.

Difference:

UDP only needs destination information because it is connectionless. TCP needs both source and destination information because it creates a connection and keeps it open for communication in both directions.

b) Why might an application developer choose UDP instead of TCP?

- UDP is faster because there is no connection setup or extra checks.
- It has less overhead because it does not do error checking, acknowledgments, or retransmission.

- It is good for real-time applications like online games, live video, or voice calls where speed is more important than perfect delivery.
- UDP can easily send data to multiple devices at once (broadcast or multicast).

c) Why is voice and video often sent over TCP nowadays?

1. **Works everywhere** – Firewalls and NATs often block UDP, but TCP (like HTTPS) is always allowed.
2. **Web support** – Streaming sites (YouTube, Netflix) use HTTP, which runs on TCP.
3. **No missing parts** – TCP ensures all packets arrive in order, so videos don't have gaps.
4. **Better streaming** – Adaptive streaming (DASH, HLS) needs TCP to switch quality smoothly.
5. **Fair sharing** – TCP has congestion control, so it shares bandwidth fairly.
6. **Easier deployment** – Developers don't need extra protocols; TCP already works with CDNs.
7. **Reliable delivery** – Even if packets are lost, TCP resends them, keeping quality high.
8. **User experience** – For stored videos, people prefer smooth playback over very low delay.

2.

Suppose Host A sends two TCP segments back to back to Host B over a TCP connection. The first segment has sequence number 90; the second has sequence number 110.

a) How much data is in the first segment?

b) Suppose that the first segment is lost but the second segment arrives at

B. In the acknowledgment that Host B sends to Host A, what will be the acknowledgment number?

c) Is it possible for an application to enjoy reliable data transfer even when the application runs over UDP? If so, how?

d) Suppose a process in Host C has a UDP socket with port number 6789. Suppose both Host A and Host B each send a UDP segment to Host C with destination port number 6789. Will both of these segments be directed to the same socket at Host C? If so, how will the process at Host C know that these two segments originated from two different hosts?

a) How much data is in the first segment?

The first TCP segment has sequence number 90.

The second TCP segment has sequence number 110.

TCP sequence numbers count bytes.

So, data in the first segment = $110 - 90 = 20$ bytes.

b) If the first segment is lost but the second segment arrives at B, what will the acknowledgment number be?

If the first segment is lost, but the second segment arrives, Host B gets data starting at sequence number 110.

But B sees that it's missing data starting at 90.

So B will send an ACK saying:

"I'm still waiting for byte 90."

So the ACK number will be 90.

c) Is it possible for an application to have reliable data transfer over UDP? If so, how?

Yes, it is possible.

UDP itself does **not** provide reliability, but the **application** can add reliability by doing things like:

- Adding sequence numbers to packets
- Acknowledging received packets
- Resending lost packets
- Checking for errors

For example, some apps or protocols use UDP and build their own error checking and retransmission system on top of it.

d) Suppose Host C has a UDP socket with port 6789, and both Host A and Host B send UDP segments to it. Will both segments go to the same socket? How will Host C know which host sent which segment?

Yes, both UDP segments will be delivered to the same socket at Host C because they have the same destination port (6789).

How does the process know they came from different places?

Each UDP packet has:

- The source IP address
- The source port number

The socket can check these to know which host sent which packet.

3.

a) In our rdt protocols, why did we need to introduce sequence numbers?

b) why did we need to introduce timers?

c) Suppose that the roundtrip delay between sender and receiver is constant and known to the sender. Would a timer still be necessary in protocol rdt 3.0, assuming that packets can be lost? Explain.

d) In protocol rdt3.0, the ACK packets flowing from the receiver to the sender do not have sequence numbers (although they do have an ACK field that contains the sequence number of the packet they are acknowledging). Why is it that our ACK packets do not require sequence numbers?

a) Why did we need sequence numbers in rdt protocols?

We use sequence numbers to:

- Keep track of each packet sent.
- Detect duplicates, so the receiver knows if a packet is new or just a retransmission.
- Make sure data is delivered in the correct order.

Without sequence numbers, the receiver wouldn't know if a packet was lost and resent — or if it was a new one.

b) Why did we need to introduce timers?

Timers are needed to handle **lost packets or ACKs**.

- If a packet or its ACK is lost, the sender would wait forever.
- A timer lets the sender wait for a certain time. If the timer runs out and no ACK arrives, the sender **retransmits** the packet.

So, timers help recover from lost data or lost acknowledgments.

c) If the roundtrip delay is constant and known, do we still need a timer in rdt 3.0?

Round Trip Time(RTT) is the total time it takes for a packet (message) to go from the sender to the receiver **and back again**.

Yes, a timer is still needed.

- Even if the round-trip time is known, **packets can still get lost**.
- If a packet or its ACK is lost, the sender will never get a response.
- So, the sender needs a timer to **retransmit** the lost packet after the expected time passes.

Without a timer, the sender would wait forever if something is lost.

d) In protocol rdt 3.0, the ACK packets do not have sequence numbers (but they do have an ACK field that contains the sequence number of the packet they acknowledge). Why do ACK packets not require sequence numbers?

ACK packets do not need their own sequence numbers because:

- They only need to tell the sender **which data packet was received correctly**.
- This is done using the **ACK field**, which carries the sequence number of the last correctly received data packet.
- The sender can figure out what to do using this field alone.

So, giving ACK packets a separate sequence number would be unnecessary because the ACK field already does the job.

4.

a) Visit the Go-Back-N interactive animation at the Companion Website.

a. Have the source send five packets, and then pause the animation before any of the five packets reach the destination. Then kill the first packet and resume the animation. Describe what happens.

b. Repeat the experiment, but now let the first packet reach the destination and kill the first acknowledgment. Describe again what happens.

c. Finally, try sending six packets. What happens?

b) Repeat R12, but now with the Selective Repeat interactive animation. How are Selective Repeat and Go-Back-N different?

Part a) Go-Back-N (GBN) Experiments

a.1) The source sends five packets numbered 0, 1, 2, 3, and 4. The first packet (packet 0) is killed before reaching the destination. Since packet 0 does not arrive, the destination cannot accept packets 1 to 4 because they arrive out of order. These packets are discarded, and the destination keeps waiting for packet 0. Meanwhile, the source does not receive any acknowledgment for packet 0. When the timer expires, the source retransmits all the packets (0, 1, 2, 3, and 4) again.

a.2) In this case, the source sends packets 0, 1, 2, 3, and 4. Packet 0 arrives at the destination, which sends an acknowledgment (ACK 1) back. However, the acknowledgment is killed before it reaches the source. The source keeps waiting for the ACK but never receives it. After the timer expires, the source assumes packet 0 was lost and resends packets 0, 1, 2, 3, and 4 again. The destination receives packet 0 again but discards it because it was already received.

a.3) When six packets are sent with a window size of five, the source first sends packets 0, 1, 2, 3, and 4. The window is now full, so packet 5 cannot be sent until at least one acknowledgment is received, such as ACK 1 for packet 0. Once an acknowledgment arrives, the window slides forward and allows new packets to be sent. If no acknowledgment arrives, the window remains full and no new packets can be transmitted.

4b) How are Selective Repeat and Go-Back-N different?

In **Selective Repeat**:

- If one packet is lost, **only that lost packet is resent**, not all packets after it.
- The receiver keeps the correct packets **out of order** and stores them until the missing one arrives.
- So, Selective Repeat is **more efficient**, because it **doesn't resend unnecessary packets**.

In **Go-Back-N**:

- If one packet is lost, the sender **goes back** and resends that packet **and all packets after it**, even if they were okay.
- The receiver **can't accept out-of-order packets** — it must get them in order.

So, **Selective Repeat saves time and bandwidth**, but it needs **more memory** to store out-of-order packets.

5.

a) Consider two hosts, A and B, transmitting a large file to a server C, over a bottleneck link with rate R . To transfer the file, the hosts use TCP with the same parameters (including MSS and RTT) and start their transmissions at the same time. Host A uses a single TCP connection for the entire file, while Host B uses 9 simultaneous TCP connections, each for a portion (i.e., a chunk) of the file. What is the overall transmission rate achieved by each host at the beginning of the file transfer? Is this situation fair?

b) Suppose Client A requests a web page from Server S through HTTP and its socket is associated with port 33000.

- a. What are the source and destination ports for the segments sent from A to S?
- b. What are the source and destination ports for the segments sent from S to A?
- c. Can Client A contact to Server S using UDP as the transport protocol?
- d. Can Client A request multiple resources in a single TCP connection?

5a)

When Hosts A and B both send a large file over a bottleneck link with rate R , they share the link's capacity.

- Host A uses **1 TCP connection**.
- Host B uses **9 TCP connections**.
- TCP shares bandwidth fairly **per connection**, not per host.
- Total active connections = 1 (Host A) + 9 (Host B) = 10.

So:

- Host A gets about **$R/10$** .
- Host B gets about **$9R/10$** .

This is not fair because Host B gets much more bandwidth just by opening more connections. TCP does not know that all nine connections belong to the same host, so it shares bandwidth equally among connections, not among users.

5b)

a. What are the source and destination ports for segments sent from A to S?

When Client A sends HTTP requests to Server S, the source port is 33000 (the port chosen by A's TCP socket) and the destination port is usually port 80 (the default port for HTTP).

b. What are the source and destination ports for segments sent from S to A?

When Server S replies to Client A, the source port in the segment is 80 (the server's HTTP port) and the destination port is 33000 (the port that A used for its socket).

c. Can Client A contact Server S using UDP as the transport protocol?

Yes, Client A can contact Server S using UDP as the transport protocol. However, this is not the typical way to do it. HTTP is a connection-oriented protocol, and UDP is a connectionless protocol. This means that each HTTP request and response would be sent as a separate UDP packet, which would add overhead and make the connection less reliable.

No, Client A cannot use UDP for standard HTTP communication. HTTP is designed to run over TCP because it requires:

- Reliable, ordered delivery of data
- Connection-oriented communication
- Error detection and retransmission

While some experimental protocols like HTTP/3 use UDP (specifically QUIC over UDP), standard HTTP/1.1 and HTTP/2 require TCP.

d. Can Client A request multiple resources in a single TCP connection?

Yes. HTTP/1.1 supports persistent connections, so multiple resources can be requested over the same TCP connection. HTTP/2 makes this even more efficient by allowing multiple requests at the same time (multiplexing) over one connection.

6.

a) UDP and TCP use 1's complement for their checksums. Suppose you have the following three 16 bit words: 0101001101100110; 0111010010110100; 0000110111000001. What is the 1's complement of the sum of these words?

Show all work. Why is it that UDP offers a checksum? With the 1's complement scheme, how does the receiver detect errors? Describe how a single bit flip can be detected.

b) Assume that a host receives a UDP segment with 01011101 11110010 (we separated the values of each byte with a space for clarity) as the checksum. The host adds the 16-bit words over all necessary fields excluding the checksum and obtains the value 00110010 00001101. Is the segment considered correctly received or not? What does the receiver do?

6(a)

Given three 16-bit words:

Word 1: 0101001101100110

Word 2: 0111010010110100

Word 3: 0000110111000001

Step 1: Add Word 1 and Word 2

```
0101001101100110
+ 0111010010110100
-----
```

1100100000011010

Step 2: Add Word 3

```
1100100000011010
+ 0000110111000001
-----
```

1101010111011011

Step 3: Take the 1's complement

To find the 1's complement, flip every bit:

1101010111011011 → 0010101000100100

So, the 1's complement of the sum is:

0010101000100100

Why does UDP offer a checksum?

UDP provides a checksum to detect errors that may occur during transmission, such as flipped bits or missing bits. The checksum helps ensure data integrity.

How does the receiver detect errors using the 1's complement scheme?

When a segment arrives, the receiver adds all 16-bit words and the checksum. If the segment is correct, the total sum should be all ones (1111111111111111). If the result is not all ones, there is an error.

How can a single bit flip be detected?

If even one bit changes during transmission, the sum will not add up to all ones. This allows the receiver to detect the error and discard the corrupted segment.

6(b)

Given:

Received checksum: 01011101 11110010

Sum of all fields excluding the checksum: 00110010 00001101

Add the checksum and the sum:

```
0011001000001101
+ 0101110111110010
-----
```

1000111111111111

There's no carry to wrap around. The result is 1000111111111111.

This is **not all 1s** (1111111111111111). Because the result is not all 1s, the segment has an error.

So, the receiver **detects that an error occurred**. In UDP, when an error is detected, the corrupted segment is **simply discarded**. UDP does not fix the error or request retransmission — that's up to the application if needed.

7.

a) Suppose that the UDP receiver computes the Internet checksum for the received UDP segment and finds that it matches the value carried in the checksum field. Can the receiver be absolutely certain that no bit errors have occurred? Explain.

b) In protocol rdt3.0, the ACK packets flowing from the receiver to the sender do not have sequence numbers (although they do have an ACK field that contains the sequence number of the packet they are acknowledging). Why is it that our ACK packets do not require sequence numbers?

c) Draw the FSM for the receiver side of protocol rdt3.0.

7(a)

No, the receiver cannot be absolutely certain that no bit errors have occurred, even if the computed checksum matches the value in the checksum field.

Reasons:

1. **Limited error detection capability:** The Internet checksum uses a simple 16-bit one's complement sum. This method can detect many single-bit errors and some multi-bit errors, but not all possible error patterns.

2. Undetected error scenarios:

- If two bits flip in such a way that they cancel each other's effect on the checksum
- Multiple bit errors that result in the same checksum value
- Errors in positions that don't affect the final sum

3. **Mathematical limitation:** With a 16-bit checksum, there are 2^{16} possible checksum values but many more possible data combinations, meaning multiple different data patterns can produce the same checksum (hash collisions).

The checksum provides reasonable error detection for typical network errors but is not foolproof.

7(b)

- **Data packets** must have sequence numbers (0, 1, 0, 1, ...) to detect **duplicates** when retransmissions happen.
- But **ACK packets only confirm receipt of a specific sequence number**.
- For example:
 - If the sender sends a packet with sequence number 0, the receiver replies with **ACK(0)**.
 - If the sender sends a packet with sequence number 1, the receiver replies with **ACK(1)**.
- So the ACK itself does not need its **own sequence number field** — instead, it carries the **expected sequence number of the next packet**.

◆ In other words:

- Sequence numbers are essential for **data packets**.
- ACK packets only need to **refer to the last correctly received data packet** (or the next expected).
- Therefore, ACKs don't require a new independent sequence number.

7c)

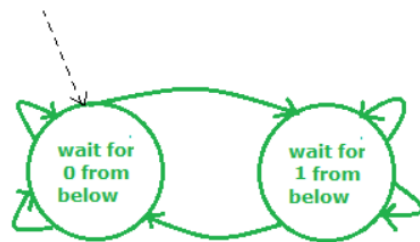
To transfer the data over a channel, rdt3.0 protocol is useful. It is a reliable protocol.

The sender transmits the packet and the receiver acknowledges it by sending an ACK and confirms the packet is received.

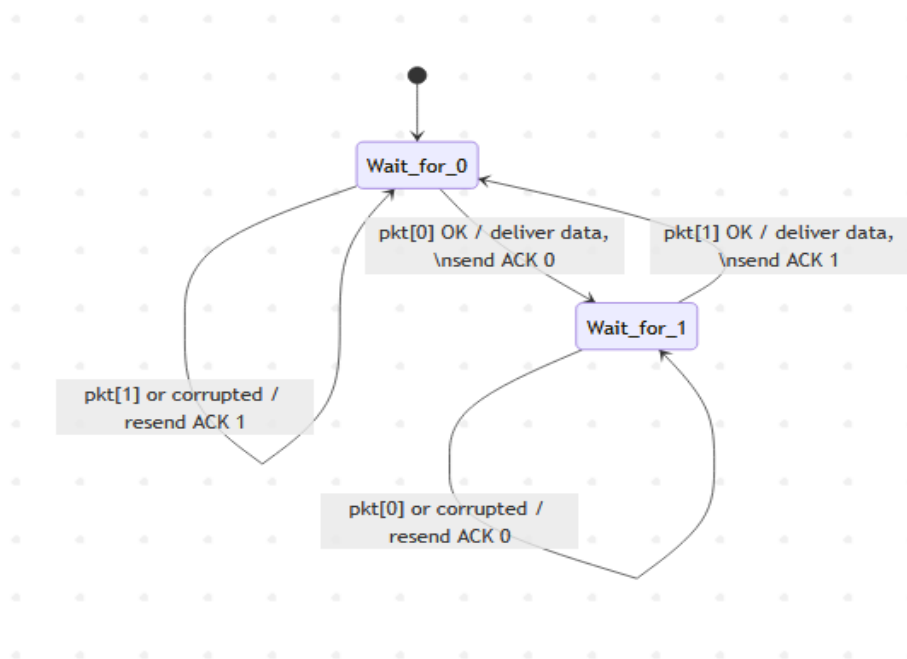
The rdt3.0 protocol allows duplicate packets into the sender-to-receiver data stream by adding timeout. This is not possible in rdt2.0.

The rdt3.0 and rdt2.0 works in the same way at receiver end. Both handles duplicate packets.

FSM diagram for the receiver side of the protocol rdt3.0 is follows:



The FSM will be same as the FSM of rdt2.1 given in the Figure 3.12 of textbook.



8

a) Consider a channel that can lose packets but has a maximum delay that is known. Modify protocol rdt2.1 to include sender timeout and retransmit. Informally argue why your protocol can communicate correctly over this channel.

b) Consider the rdt2.2 receiver in Figure 3.14, and the creation of a new packet in the self-transition (i.e., the transition from the state back to itself) in the Wait-for-0-from-below and the Wait-for- 1-from-below states: `sndpkt=make_pkt(ACK, 1 ,checksum)` and `sndpkt=make_pkt(ACK,0,checksum)`. Would the protocol work correctly if this action were

removed from the self-transition in the Wait-for-1 -from- below state? Justify your answer. What if this event were removed from the self-transition in the Wait-for-0-from-below state?

a) rdt2.1 with Timeout (Loss Handling)

Modification:

- Add a **timer** at sender. If no ACK arrives before $\text{TIMEOUT} > \text{max RTT}$, sender **retransmits the packet**.

Sender:

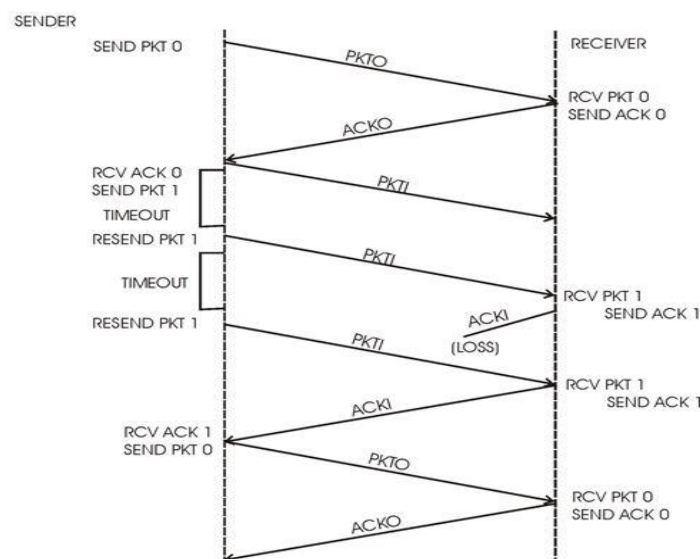
- Sends packet with sequence number and starts timer.
- On correct ACK → **stop timer**, move to next packet.
- On timeout → **retransmit same packet**.

Receiver:

- If packet is correct & expected → **deliver to application**, send ACK.
- If duplicate or corrupt → **discard**, resend last ACK.

Correctness & Features:

- **Reliable delivery:** Lost or corrupted packets are retransmitted until acknowledged.
- **In-order delivery:** Only expected sequence number is accepted; duplicates ignored.
- **Progress guaranteed:** Timer ensures sender retries even if ACK is lost.
- **Deadlock-free:** Sender always eventually sends next packet.
- Works over channels with **bounded delay**, ensuring **eventual successful communication**.



b)

Background:

In **rdt2.2**, the receiver has two states:

1. **Wait-for-0-from-below (expecting seq 0)**
2. **Wait-for-1-from-below (expecting seq 1)**

Self-transition: occurs when the receiver gets a **corrupted packet or duplicate packet**.

- In the original design, the receiver **resends the ACK** of the last correctly received packet (`sndpkt = make_pkt(ACK, last_seq, checksum)`) during this self-transition.

This ensures the **sender knows the receiver got the last correct packet** even if an ACK or data was lost/corrupted.

Question Part 1: Removing `sndpkt=make_pkt(ACK,0,checksum)` in Wait-for-1-from-below

- If the receiver **does not resend the ACK** upon a duplicate/corrupt packet:
 - Sender may **never get the ACK** for the last correctly sent packet.
 - If that ACK was lost in the network, the sender **times out and retransmits**.
 - Retransmission may still work correctly (because rdt2.2 handles retransmission), but the **sender may retransmit unnecessarily**.

Answer: The protocol would **still work correctly**, but it becomes **less efficient**—extra retransmissions may happen.

Question Part 2: Removing `sndpkt=make_pkt(ACK,1,checksum)` in Wait-for-0-from-below

- If the receiver does not resend the ACK:
 - Suppose the sender sends a packet with seq 0, receiver gets it, sends ACK0, but the **ACK0 is lost**.
 - Sender retransmits seq 0.
 - Receiver is **expecting seq 0**, but now sees **duplicate seq 0**.
 - If it **does not resend ACK0**, sender **never receives acknowledgment**, and cannot proceed.

Answer: The protocol **would fail** if the self-transition action is removed in **Wait-for-0-from-below**, because the sender may get stuck waiting for an ACK.

9.

a) Assume Host A is streaming a video from Server B using UDP. Also assume that the network suddenly becomes very congested while Host A is seeing the video. Is there any way to handle this situation with UDP? What about with TCP? Is there any other option?

b) Consider a stop-and-wait data-transfer protocol that provides error checking and retransmissions but uses only negative acknowledgments. Assume that negative acknowledgments are never corrupted. Would such a protocol work over a channel with bit errors? What about over a lossy channel with bit errors?

a) Handling Network Congestion

UDP Handling of Congestion:

UDP provides no congestion control mechanism. When network becomes congested:

- No rate adaptation - continues sending at same rate
- Packet loss increases with no feedback mechanism
- Application must implement its own congestion handling (adaptive streaming, buffer management)

TCP Handling of Congestion:

TCP has built-in congestion control:

- Congestion window reduction - automatically reduces sending rate
- Slow start and fast retransmit/recovery mechanisms
- Fair bandwidth sharing among connections

Other Options:

Modern systems use **QUIC (HTTP/3)**, which runs over UDP but **adds reliability, congestion control, and encryption** like TCP.

b) Stop-and-Wait with Negative ACKs Only

Protocol Description: (1 mark)

- Send packet, wait for response
- Only NACKs sent for corrupted packets, no positive ACKs
- Requires timeout mechanism for lost packet detection

Channel with Bit Errors Only: (2 marks)

Yes, protocol would work but inefficiently:

- Sender waits for timeout on every successful transmission

- NACK sent only for corrupted packets
- Much slower than positive ACK systems due to timeout delays

Lossy Channel with Bit Errors: (2 marks)

Protocol becomes fundamentally unreliable:

Major problems:

- Lost NACK problem: If NACK is lost, sender assumes successful delivery
- Timeout confusion: Cannot distinguish between successful delivery, lost packet, or lost NACK
- Failure scenario: Corrupted packet + lost NACK = corrupted data accepted as valid

Conclusion: Protocol fails its primary purpose in lossy environments - positive ACKs with sequence numbers needed for reliability.

10. a) Consider the GBN protocol with a sender window size of 4 and a sequence number range of 1024. Suppose that at time t , the next in-order packet that the receiver is expecting has a sequence number of k . Assume that the medium does not reorder messages. Answer the following questions:

- What are the possible sets of sequence numbers inside the sender's window at time t ? Justify your answer.**
- What are all possible values of the ACK field in all possible messages currently propagating back to the sender at time t ? Justify your answer.**

b) Give one example where buffering out-of-order segments would significantly improve the throughput of a GBN protocol.

c) Consider a scenario where the three hosts A, B, and C are connected as a ring: A to B, B to C, and C to A. Assume that A and C run protocol rdt3.0, whereas B simply relays all messages received from A to C.

- Does this arrangement enable reliable delivery of messages from A to C?**
- Can B tell if a certain message has been correctly received by A?**

a)

a.a) Possible sets of sequence numbers inside the sender's window at time t

The receiver is expecting the next in-order packet with sequence number k . This implies that the sender has successfully sent all packets up to $k-1$, but has not yet received acknowledgments for them. Given that the sender window size is 4, the sender can have at most 4 unacknowledged packets outstanding. Therefore, the possible sequence numbers inside the sender's window are:

$$\{k, k+1, k+2, k+3\}(\text{modulo } 1024)$$

In other words, the base sequence number of the sender's window is k , and the next sequence numbers to send belong to the set $\{k, k+1, k+2, k+3\}$.

b.b) Possible values of the ACK field in ACKs propagating back to the sender

The receiver sends cumulative acknowledgments indicating the highest in-order packet received correctly. Since the receiver expects packet k , it means it has correctly received all packets up to $k - 1$. However, due to network delay, acknowledgments for previously received packets—namely, $k - 1$, $k - 2$, $k - 3$, or $k - 4$ might still be in transit to the sender. Hence, the possible ACK values in flight are:

$$\{k, k-1, k-2, k-3, k-4\}(\text{modulo } 1024)$$

These values correspond to acknowledgments for the last correctly received in-order packets.

b)

Example: Buffering out-of-order segments in GBN (Go-Back-N)

Scenario:

- Sender transmits packets: 0, 1, 2, 3, 4, 5 in sequence.
- Assume **packet 1 is lost**, but packets 2, 3, 4, 5 arrive at the receiver.

Without buffering (standard GBN):

- Receiver discards all out-of-order packets (2, 3, 4, 5) until it receives the missing packet 1.
- Sender must **retransmit packet 1 and all subsequent packets**.
- Throughput drops significantly due to repeated retransmissions.

With buffering out-of-order segments:

- Receiver **stores packets 2, 3, 4, 5** temporarily.
- Once missing packet 1 arrives, the receiver can **deliver all buffered packets in order** immediately.
- Reduces unnecessary retransmissions and **improves throughput**.

C) Scenario: Ring network ($A \rightarrow B \rightarrow C \rightarrow A$)

c.a) Yes, this arrangement enables reliable delivery of messages from A to C because both A and C use the rdt3.0 protocol, which handles lost, corrupted, and duplicate packets using sequence numbers, acknowledgements, and timeouts. Host B only forwards packets and does not interfere with the reliability mechanisms.

c.b) No, B cannot know if A got the message correctly. B just forwards the packets and does not check or keep track of ACKs or sequence numbers. Only A and C handle the acknowledgments to make sure the message is delivered correctly.